

CSIS 2300 Introduction to Algorithms

Spring 2026 – Exam 1 Solutions

(1) [5 points each] For each of these recurrence relations, give its asymptotic big- Θ complexity class. (Your answers should be something like $T(n) = \Theta(n \log n)$). Justify your responses. If you use the Master Theorem, be sure to specify the values of a , b , and d .

$$(a) T(n) = T(n - 2) + 3n^2 + 10n$$

This problem cannot be solved using the Master Theorem, because the recursion is not being cut down by a fraction each time.

Instead, we recognize that single calls of $T(n - 2)$ produce a total of $n/2$ recursive calls. Each one requires $\Theta(n^2)$ work on average, resulting in $\Theta(n^3)$ overall.

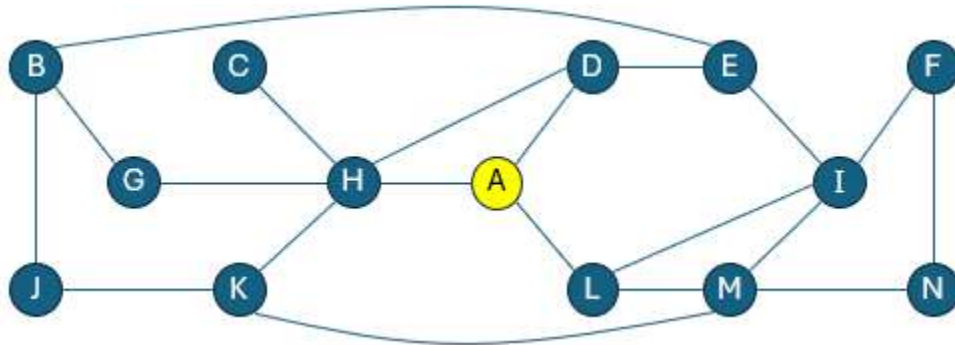
$$(b) T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$a = 4, b = 2, d = 2$. Since $a = b^d$, the recursion and the non-recursive work “balance” each other to some extent, and we get $\Theta(n^2 \log n)$.

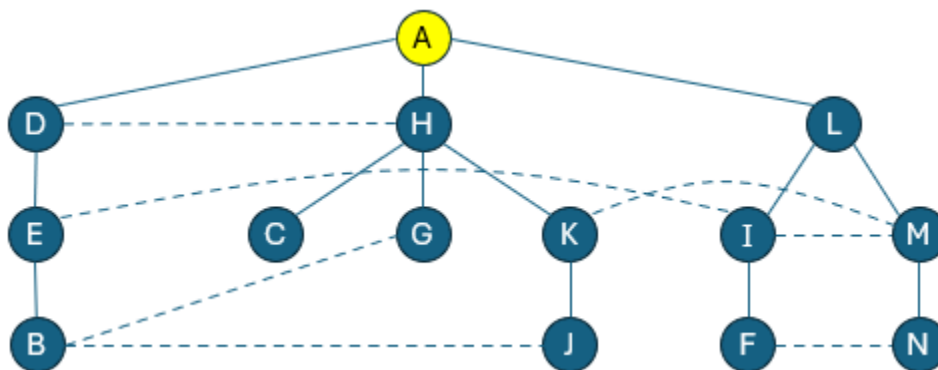
$$(c) T(n) = 2T\left(\frac{n}{3}\right) + 3n$$

$a = 2, b = 3, d = 1$. Since $a < b^d$, the non-recursive work dominates the actual recursion, and we get $\Theta(n)$.

(2) (a) [10 points] Execute a breadth-first search on the undirected graph below, starting with node a and breaking all ties alphabetically. Draw the search tree using solid lines for the tree edges and clearly indicating which nodes are at each level of the tree. Then add the cross edges using dashed lines.



Solution:



(b) [10 points] Explain why all cross edges in a breadth-first search tree for an undirected graph connect nodes that are at most one level apart.

The key insight is that levels in a BFS tree represent distance (number of edges in the path) from the source node. If there were a cross edge between a node u on layer k and a node v on layer $k + 2$ (or more than 2), then there would be a path from the source to v of length $k + 1$ that passed through u , which means that v is in the wrong layer.

(3) [8 points each] Prove each of the following.

(a) $n \cdot 2^n = \omega(2^n)$

Method 1, limit test:

$$\lim_{n \rightarrow \infty} \frac{n \cdot 2^n}{2^n} = \lim_{n \rightarrow \infty} n = \infty$$

Note here that you must cancel the 2^n factors first; you may not cancel $\frac{\infty}{\infty}$.

Method 2, using definition of ω :

We must prove $\forall c > 0, \exists n_0$ such that $\forall n > n_0, n \cdot 2^n > c \cdot 2^n$. This is quite straightforward – since c is always positive, we can simply select $n_0 = c + 1$. (Anything larger than c will work.)

(b) For this one, **do not** use the Limit Test.

$$\sum_{i=1}^n (\log i) = O(n \log n)$$

Since we are barred from using the limit test, we must specify the constants c and n_0 from the definition of big-O. Specifically, we need to find some $c > 0, n_0 > 0$ such that $\forall n > n_0, \sum_{i=1}^n (\log i) \leq cn \log n$.

In this case, the key insight is that we can rewrite the right side as a sum, giving $\sum_{i=1}^n (\log i) \leq c \sum_{i=1}^n \log n$. Since i is always $\leq n$, this is true for any $c \geq 1$. Thus we can pick, for example, $c = 1, n_0 = 1$. (Any greater values work. I prefer to use $n_0 = 2$, just because $\log 1 = 0$.)

(4) [5 points each] Analyze the following pseudocode, assuming the initial call is $\text{Moe}(A, 0, n-1)$, where A is an array of n integers:

```
def Moe(A, L, R):
    if A[L] > A[R]:
        swap(A, L, R) # switch elements A[L] and A[R]
    size = R - L + 1
    if size >= 3:
        t = size // 3 # integer division
        Moe(A, L, R-t)
        Moe(A, L+t, R)
        Moe(A, L, R-t)
```

- (a) Identify the critical operation.
- (b) Write a recurrence relation for the runtime.
- (c) Solve the recurrence relation (i.e. give the closed-form asymptotic runtime).
- (d) What does this algorithm appear to do? (Try it on a small array: [5 9 3 0 1 4])
Would you consider using it for that purpose?

(a) The critical operation is the comparison on line 2. The swap doesn't always happen, and the size calculation and comparison on lines 4 & 5 are faster operations because they don't involve an array access.

(b) Key insight: since t is $1/3$ of size, the recursive calls operate on $2/3$ of the original array. Also, everything else is a constant time operation. $T(n) = 3T\left(\frac{2}{3}n\right) + O(1)$

(c) Master Theorem: $a = 3, b = \frac{3}{2}, d = 0$. Since $a > b^d$, the recursion dominates this process and we get $\Theta(n^{\log_{1.5} 3})$.

(d) The algorithm sorts the list by sorting the first $2/3$ of the list, then the last $2/3$ of the list, then the first $2/3$ of the list again.

An exact value for $\log_{1.5} 3$ is not required – we can simply note that it must be more than 2 (since $1.5^2 = 2.25$). As this sort is worse than quadratic time, we should not use it.

(5) You are given an array of n arbitrary integers (positive, negative, or zero; repeats allowed). You wish to find the run of consecutive values in the array that has the highest sum. For example, in the array $[-3 \ 5 \ -8 \ -1 \ \underline{3 \ 7 \ 1 \ 0 \ 6 \ -4 \ 5}] \ -9 \ 0 \ -2 \ 4 \ 1 \ -2]$, the best such subarray is the underlined section, with a total sum of 18. (Note: Both the *empty* subarray (when all values are negative) and the full array itself (when all are positive) are possible answers to this problem.)

(a) [10 points] Give a brute force algorithm for solving this problem. State the runtime of your algorithm, explaining your answer.

List every subsequence – there are $\binom{n}{2}$ of these, as the endpoints can be any pair of values. For each one, add it up. This requires linear time on average. Thus, the full algorithm is $\Theta(n^3)$.

Note there is an easy update to get this to $\Theta(n^2)$ – as you generate the subsequences that start with each value, keep a running sum. This avoids some re-adding of values that you've already added up.

(b) [20 points] Devise and clearly state a divide-and-conquer algorithm for solving this problem. (You may use English or pseudocode.) Your algorithm should have $O(n \log n)$ runtime. (Hint: It's more like mergesort than like quicksort; an approach similar to the closest-pair problem we discussed in class will work well.)

Divide the list in half and recursively find the best subsequence sum in each half – call these sums A & B. Let C represent the best subsequence sum that includes elements from both halves (i.e. subsequences that bridge the gap). We should return the maximum of A, B, and C; the trick is to calculate C.

We can find C by recognizing that the best subsequence that crosses the halfway point must include the elements on either side of the gap. From there we can explore to the left using a running sum to find the best left endpoint, and do the same on the right. This process takes linear time.

The overall recurrence relation is $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$ – the same one as for mergesort and quicksort, which we know are $\Theta(n \log n)$.

(blank page for additional work, if needed; please do not detach)